

Sistemas Distribuídos (DCC/UFRJ)

Módulo 4 - Parte 3 - Aula 1: Comunicação orientada a mensagem

Profa. Silvana Rossetto

- 1 Caracterização da **comunicação orientada a mensagens**
- 2 Comunicação em grupo
- 3 Exemplos de **middleware orientado a mensagens**:
 - interface de passagem de mensagem (MPI)
 - fila de mensagens (MQ)
 - eventos e sistemas *publish/subscribe*
- 4 Comunicação de fluxos de dados

Oferece **serviços adicionais às primitivas básicas de troca de mensagens** (send/receive)

- diferente de RPC, **não visa ocultar a troca de mensagens** entre processos, mas também oferece abstrações para facilitar a programação
- geralmente provê **comunicação persistente** (não requer emissor e receptor ativos ao mesmo tempo)
- é oferecida por um **middleware orientado a mensagem** (*Message-Oriented Middleware (MOM)*)

Um processo pode **enviar uma mensagem para um grupo de processos**

- *broadcast*: quando a mensagem é enviada para **todos os demais processos**
- *multicast*: quando a mensagem é enviada para um **subgrupo de processos**

Usada para:

- enviar requisições para um conjunto de réplicas de servidores
- realizar descoberta de serviços
- notificar eventos

Message Passing Interface (MPI) é um exemplo de MOM voltado para **programação paralela**

- adota **passagem de mensagem transiente** (assume que falhas são fatais)
- oferece **send/receive síncrono e assíncrono**
- oferece facilidades de **comunicação em grupo** (ex., enviar dados ou receber resultados de um grupo de processos)

Requisitos para MPI:

- abstrações de comunicação convenientes para a programação paralela
- implementação das abstrações com sobrecarga mínima
- independência de plataformas de hardware e software
- possibilidade de diferentes implementações (ex., MPICH, OpenMPI, LAM)

Message Queuing System (MQ) é outro exemplo de MOM

- provê serviço de **comunicação persistente**
- as **filas de mensagens armazenam as mensagens** até que o receptor as extraia para processamento
- a **comunicação é normalmente assíncrona**
- não há garantia sobre quando a mensagem será processada (e se será processada) pelo receptor (**desacoplamento temporal**)

Primitivas básicas para comunicação das mensagens:

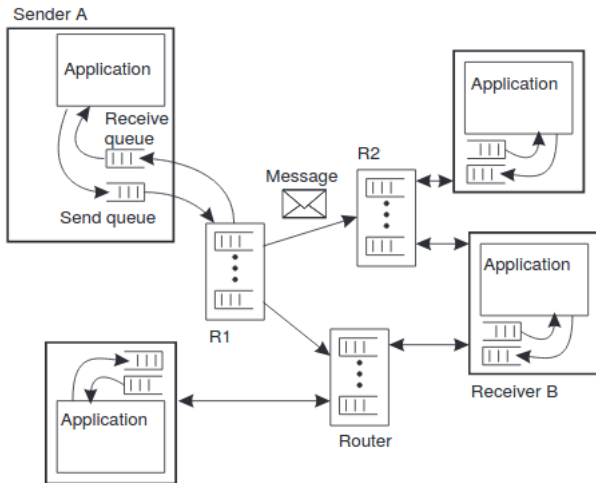
- put: acrescenta uma mensagem (**com o endereço da fila de destino**) na fila de saída da aplicação
- get: bloqueia até a fila de entrada da aplicação ter alguma mensagem e a extrai
- poll: checa se há mensagem na fila de entrada (nunca bloqueia)
- notify: define uma função que será chamada quando uma mensagem chegar

Exemplo: sistemas de fila de mensagens

O subsistema de comunicação é responsável por transferir as mensagens entre as filas:

- 1 transmitindo as mensagens da fila de envio diretamente para a máquina responsável pela fila de recepção
- 2 ou de forma mais elaborada, transmitindo as mensagens por uma rede de roteadores de mensagens

Exemplo de rede sobreposta de roteadores de mensagens



1

¹Fonte: I. Kuz, F. Rauch, M. M. T. Chakravarty e G. Heiser, Notas de Aula (COMP9243 - Week 2).

Exemplo: comunicação baseada em eventos

Normalmente associada a **sistemas**
publish/subscribe:

- o **emissor publica eventos** (que carregam atributos/dados)
- o **receptor registra interesse em certos tipos de eventos**
- o **MOM é responsável por entregar os eventos** de interesse para os receptores

Emissores e receptores estão referencialmente desacoplados **desacoplamento espacial**

Abstrações específicas para comunicações continuadas no tempo

- dados na forma de fluxos contínuos, ao invés de pedaços de conteúdo (comunicação discreta)
- há um relacionamento temporal entre os dados (além do relacionamento espacial)
- requisitos de tempo precisam ser atendidos (comunicação isócrona)

M. van Steen and A.S. Tanenbaum, “Distributed Systems”, 3rd ed., www.distributed-systems.net, 2017

- seção 4.3 (pág 204-206; 206-210)

I. Kuz, F. Rauch, M. M. T. Chakravarty e G. Heiser, Notas de Aula (COMP9243 - Week 2 (10s1)), *The University of New South Wales, School of Computer Science and Engineering*:

- seção “**Communication**”